Course: AEP CS - Application and Web Application Security

Lesson-End Project Report: Vulnerability assessment and Penetration Testing (VAPT)

Student Name: Rachita Thakkar

This report documents a security assessment conducted on the **VulnVMa Web Application**. According to me, the objective of this project was to simulate real-world hacking scenarios by performing security reconnaissance, uncover hidden directories, find configuration flaws, and intercept web traffic to collect secret by using open-source tools and manual techniques to uncover hidden flags planted by developers, as they often leave “backdoors” configuration.

This project helped me understand that the purpose of VAPT is to ensure that code left behind for convenience doesn’t become a security liability.

Target System: VulnVMa Web Application

Target IP Address: 192.168.56.3

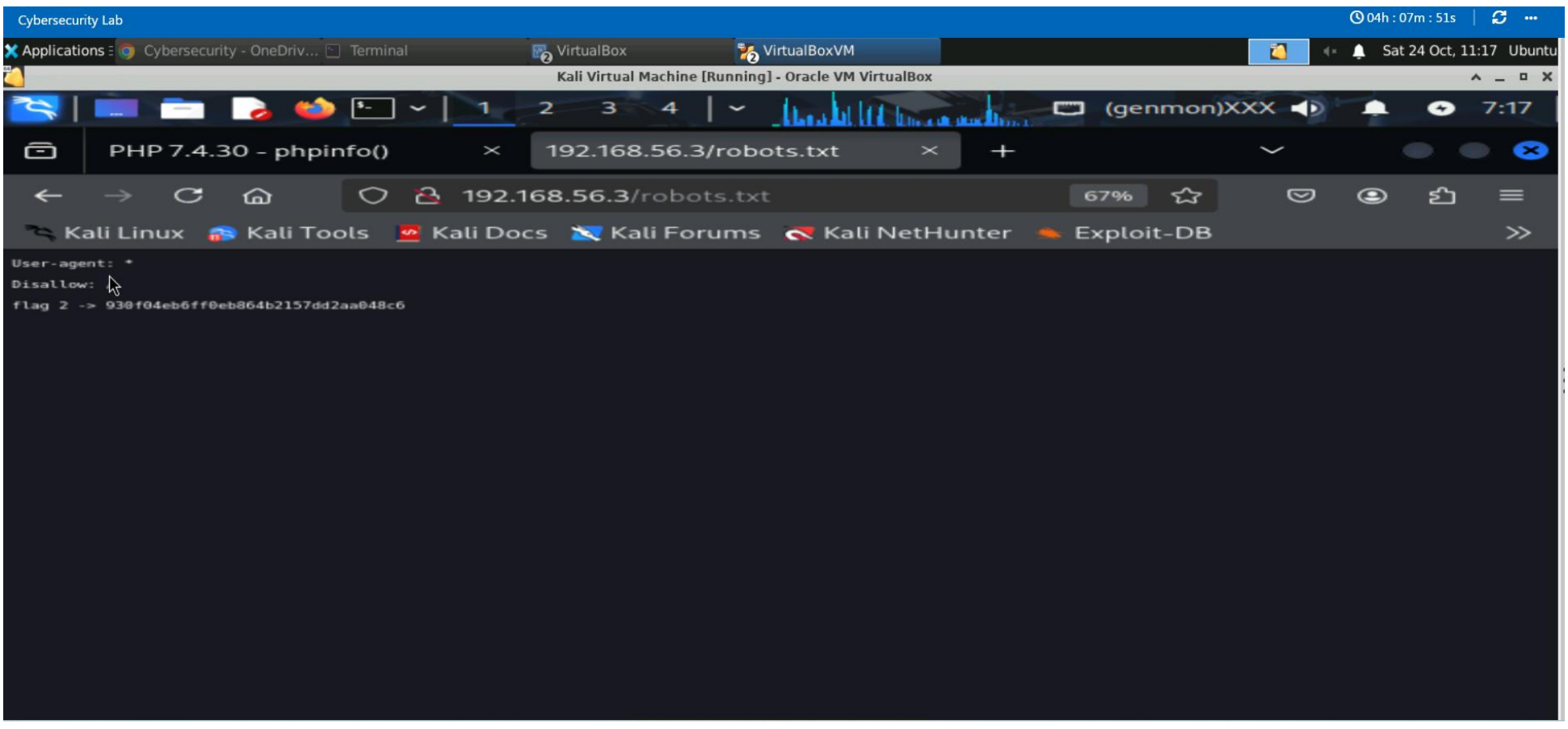
Attacker IP Address: 192.168.56.5 (Kali linux)

Environment: Oracle VM VirtualBox (Isolated Host-Only Network)

Step 1: Checking the robots.txt file (flag 2)

Inside Kali Linux I opened firefox and typed the URL **[http://192.168.56.3/robots.txt]** to inspect the web server’s public instruction file, where I found Flag 2 exposed in a plain text. “robots.txt” tells search engines like google to what not to index, however attackers check such files first because listing secret directories in a public file actually tells everyone where the hidden stuff is.

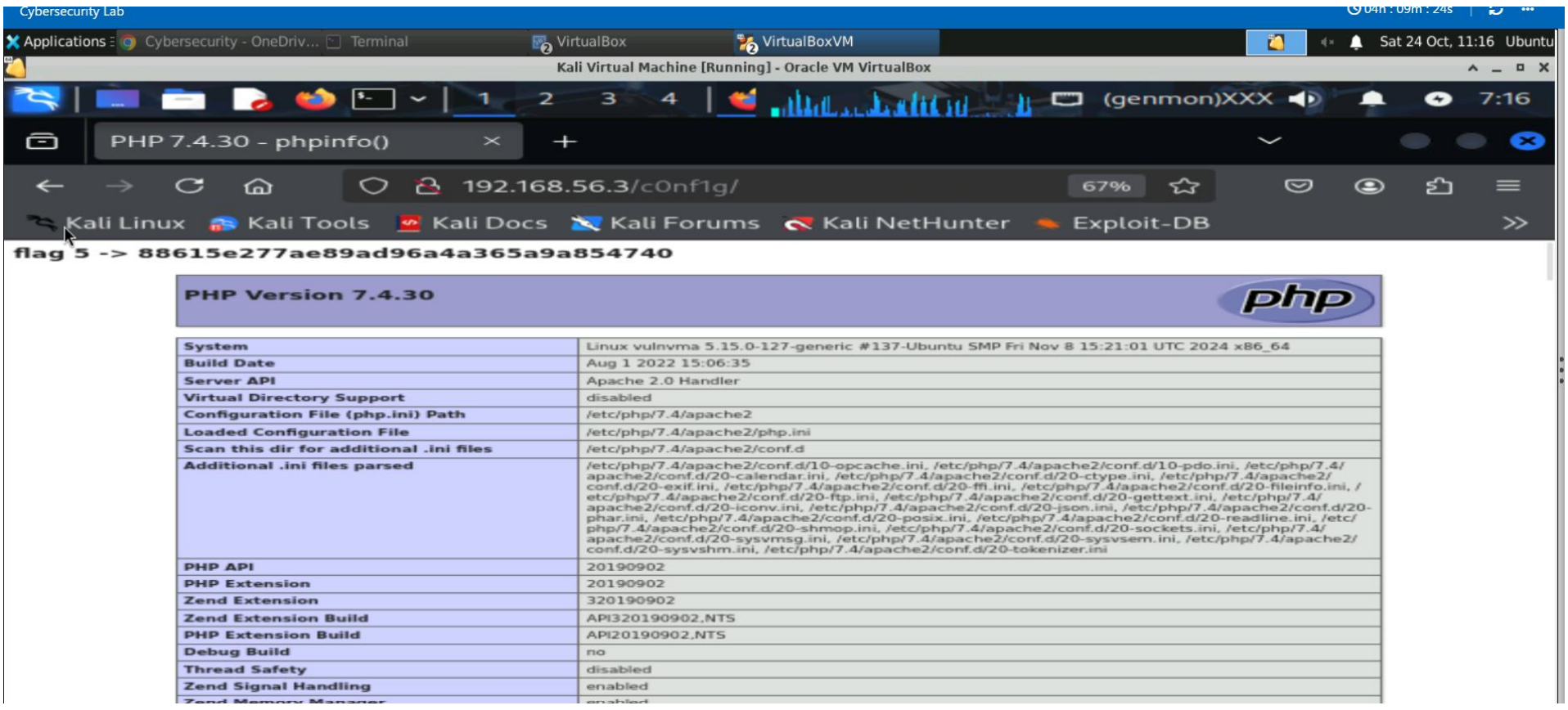
Since we achieved a Flag 2 over there, it proves that there is information leakage i.e the web server is leaking its internal secrets to public. I also learned here that “Hiding” is not “Securing”.



Step 2: Investigating Hidden Configuration Folders (Flag 5)

Again inside Kali Linux, in firefox I added the URL **[http:// 192.168.56.3/c0nf1g/]** to find the hidden configuration directory and the server loaded a large purple configuration page **“phpinfo()”** (which is a live server tool) exposing flag 5, which proves critical security vulnerability, as a real attacker would instantly get all the intelligence from the application they need to target the system.

Hackers rarely guess how to break a website, instead they look for exact version of the numbers of the software running on the server. Once they see number like PHP, they go to public database such as google and search for “exploits for php xxx”. If a known flaw exist they can use it to crash the system.

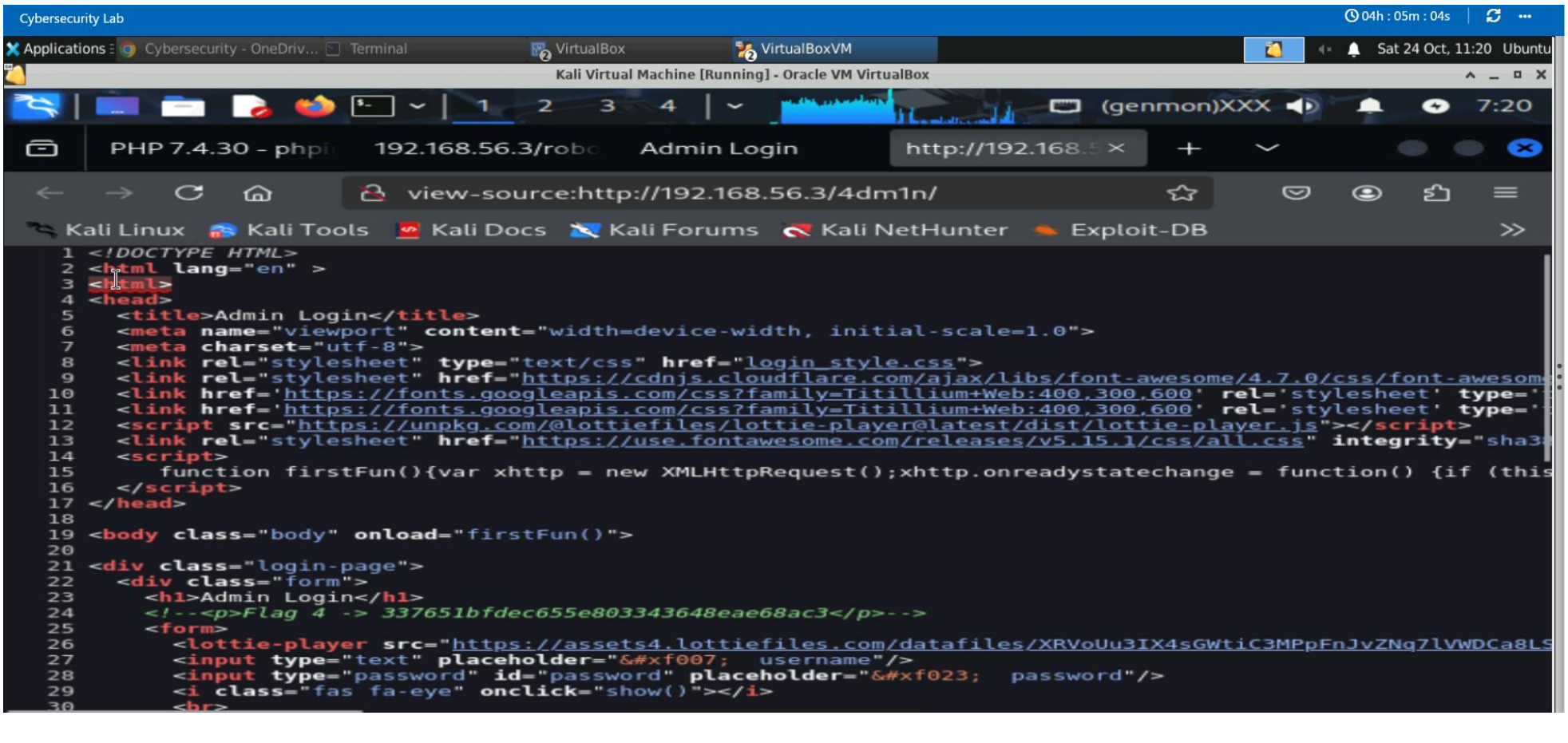


This setup page shows sensitive details about the server's backend environment and programming version.

Step 3: Auditing Front-End Source Code Comments (Flag 4)

I opened Admin login portal through firefox in kali linux with the URL **[http://192.168.56.3/4dm1n/]**, upon the page when you right click you find a “view page source” and there I saw Flag 4.

Developers often leave temporary notes or passwords inside their code while testing. Because everything in front-end HTML is downloaded straight to the user's browser, anyone can view it. I learned that developers must always clean up and delete their comments before publishing an app.

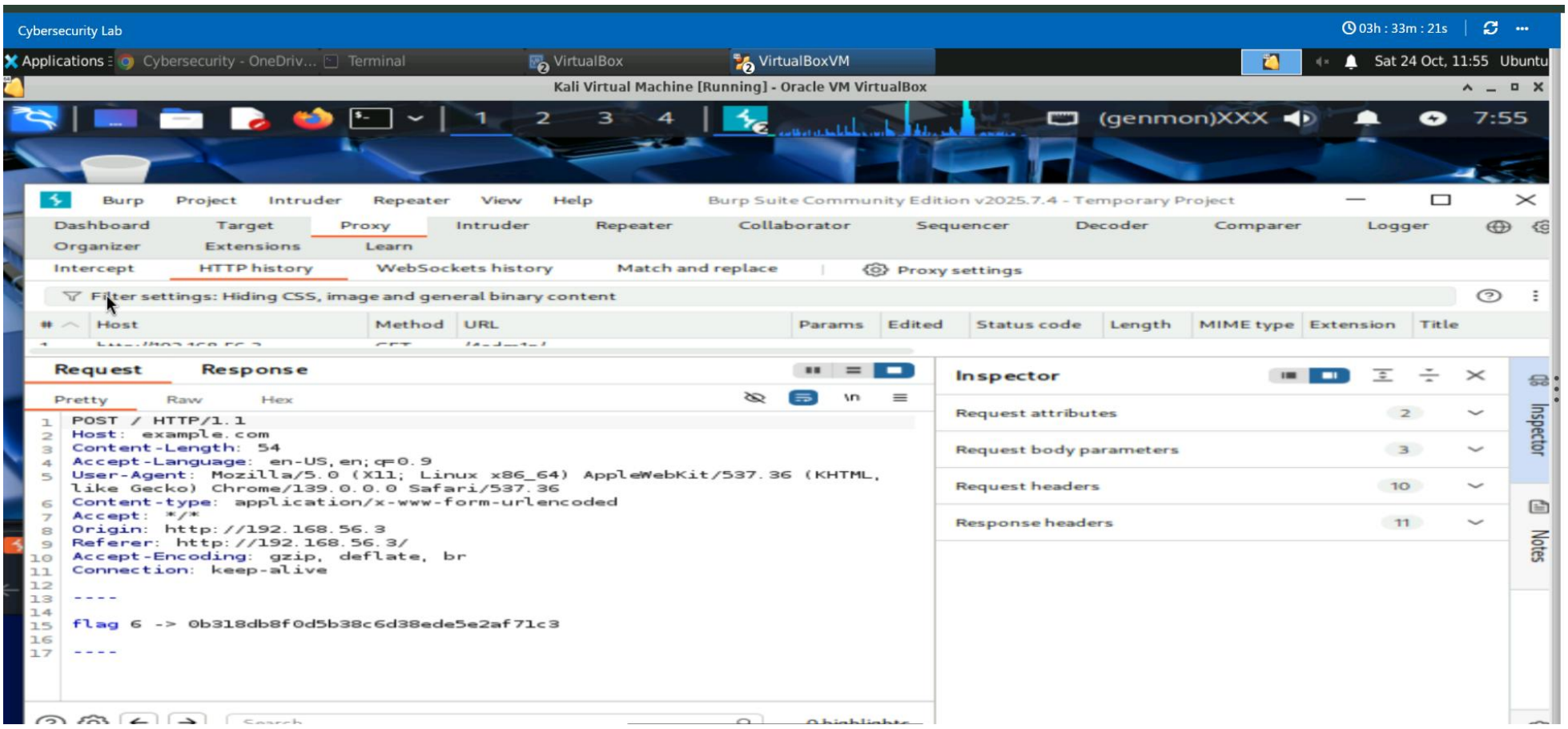


Step 4: Intercepting Login Traffic with Burp Suite (Flag 6)

I launched a professional security tool called Burp Suite and turned on traffic interception. I opened Burp's built-in browser, went to the login screen, typed a dummy username/password, and clicked submit.

What I found is Burp Suite froze the outbound request package mid-air. I looked inside the HTTP History tab under the POST request parameters and found Flag 6 hidden at the bottom.

Flag 6 determines that the network data is not private or encrypted, rather it is plain text and that every information is visible. As anything sent from a user's browser can be captured and read if it is sent in plain text. I learned that web applications need to encrypt data in transit and never trust that background parameter tokens will stay hidden.



Conclusion:

This project taught me the basics of web penetration testing. I learned that security is not just about stopping huge attacks, but also about keeping minor things locked down—like hiding folder names, deleting code notes, and ensuring that private backend details are never accidentally exposed to a regular web browser.